

Lista de Exercícios 4 — Python

Funções (def, return, parâmetros, escopo local/global, recursão)

Cada exercício deve ser resolvido em um arquivo Python separado. O nome do arquivo segue o padrão `lista4exercicioN.py`, onde N é o número do exercício (ex: `lista4exercicio1.py`, `lista4exercicio2.py`, ..., `lista4exercicio60.py`).

Todos os exercícios desta lista devem ser resolvidos **obrigatoriamente utilizando funções** (instrução `def`), mesmo quando isso não for explicitamente repetido no enunciado. Utilize `input()` para capturar os dados de teste no bloco principal do programa e chame a(s) função(ões) criada(s) para exibir o resultado.

PARTE 1 — (Exercícios 1 a 20)

Funções simples, sem parâmetros ou com poucos parâmetros, apenas com `print()` e/ou `return`.

Exercício 01

Arquivo: `lista4exercicio1.py`

Crie uma função `saudacao()`, sem parâmetros, que imprima na tela a mensagem “Olá, mundo!”. Chame a função no final do programa.

Exercício 02

Arquivo: `lista4exercicio2.py`

Crie uma função `exibir_nome(nome)` que receba um nome e imprima “Seja bem-vindo(a), <nome>!”.

Exercício 03

Arquivo: `lista4exercicio3.py`

Crie uma função `soma(a, b)` que retorne a soma de dois números.

`soma(2, 9) == 11`

`soma(7, 8) == 15`

Exercício 04

Arquivo: `lista4exercicio4.py`

Crie uma função `subtracao(a, b)` que retorne a diferença entre dois números ($a - b$).

Exercício 05

Arquivo: lista4exercicio5.py

Crie uma função `multiplicacao(a, b)` que retorne o produto de dois números.

Exercício 06

Arquivo: lista4exercicio6.py

Crie uma função `divisao(a, b)` que retorne o resultado da divisão de a por b (utilize float).

Exercício 07

Arquivo: lista4exercicio7.py

Crie uma função `dobro(n)` que retorne o dobro do número recebido.

Exercício 08

Arquivo: lista4exercicio8.py

Crie uma função `triplo(n)` que retorne o triplo do número recebido.

Exercício 09

Arquivo: lista4exercicio9.py

Crie uma função `metade(n)` que retorne a metade do número recebido.

Exercício 10

Arquivo: lista4exercicio10.py

Crie uma função `quadrado(n)` que retorne o quadrado do número recebido.

`quadrado(4) == 16`

`quadrado(9) == 81`

Exercício 11

Arquivo: lista4exercicio11.py

Crie uma função `cubo(n)` que retorne o cubo (n^3) do número recebido.

Exercício 12

Arquivo: lista4exercicio12.py

Crie uma função `media(a, b)` que retorne a média aritmética entre dois números.

Exercício 13

Arquivo: lista4exercicio13.py

Crie uma função `eh_par(n)` que retorne `True` se o número for par e `False` caso contrário.

```
eh_par(8) == True  
eh_par(7) == False
```

Exercício 14

Arquivo: lista4exercicio14.py

Crie uma função `eh_impar(n)` que retorne `True` se o número for ímpar e `False` caso contrário.

Exercício 15

Arquivo: lista4exercicio15.py

Crie uma função `area_retangulo(base, altura)` que retorne a área de um retângulo.

Exercício 16

Arquivo: lista4exercicio16.py

Crie uma função `perimetro_retangulo(base, altura)` que retorne o perímetro de um retângulo.

Exercício 17

Arquivo: lista4exercicio17.py

Crie uma função `celsius_para_fahrenheit(c)` que retorne a temperatura convertida ($F = C * 9/5 + 32$).

Exercício 18

Arquivo: lista4exercicio18.py

Crie uma função `fahrenheit_para_celsius(f)` que retorne a temperatura convertida ($C = (F - 32) * 5/9$).

Exercício 19

Arquivo: lista4exercicio19.py

Crie uma função `exibir_linha(caractere, quantidade)` que imprima o caractere repetido a quantidade de vezes indicada (ex: `exibir_linha("*", 10)` imprime `*****`).

Exercício 20

Arquivo: lista4exercicio20.py

Crie uma função `maior_de_idade(idade)` que retorne `True` se a idade for maior ou igual a 18, e `False` caso contrário.

PARTE 2 (Exercícios 21 a 30)

Funções com parâmetros opcionais (valores padrão), parâmetros nomeados e pequenas estruturas condicionais/repetição dentro da função.

Exercício 21

Arquivo: `lista4exercicio21.py`

Crie uma função `saudacao_personalizada(nome, saudacao="Olá")` que imprima “,<nome>!”. Teste chamando a função com e sem o segundo parâmetro.

Exercício 22

Arquivo: `lista4exercicio22.py`

Crie uma função `potencia(base, expoente=2)` que retorne a base elevada ao expoente (o padrão deve calcular o quadrado quando o expoente não for informado).

Exercício 23

Arquivo: `lista4exercicio23.py`

Crie uma função `calcular_desconto(valor, percentual=10)` que retorne o valor final após aplicar o percentual de desconto (padrão de 10%).

Exercício 24

Arquivo: `lista4exercicio24.py`

Crie uma função `exibir_tabuada(numero, limite=10)` que imprima, usando um `for` dentro da função, a tabuada do número recebido de 1 até o limite informado (padrão até 10).

Exercício 25

Arquivo: `lista4exercicio25.py`

Crie uma função `area_circulo(raio, pi=3.14)` que retorne a área de um círculo (área = $\pi * \text{raio}^2$).

Exercício 26

Arquivo: `lista4exercicio26.py`

Crie uma função `saudacao_por_hora(hora)` que retorne “Bom dia”, “Boa tarde” ou “Boa noite” conforme a hora informada (use `elif` dentro da função).

Exercício 27

Arquivo: lista4exercicio27.py

Crie uma função `maior_valor(a, b, c)` que retorne o maior dos três números recebidos.

Exercício 28

Arquivo: lista4exercicio28.py

Crie uma função `menor_valor(a, b, c)` que retorne o menor dos três números recebidos.

Exercício 29

Arquivo: lista4exercicio29.py

Crie uma função `classificar_nota(nota)` que retorne o conceito do aluno usando `elif` dentro da função: A (`nota >= 9`), B (`nota >= 7`), C (`nota >= 5`) e D (`nota < 5`).

Exercício 30

Arquivo: lista4exercicio30.py

Crie uma função `imprime_triangulo(linhas)` que utilize um `for` interno para desenhar um triângulo de asteriscos com a quantidade de linhas informada (linha 1 com 1 asterisco, linha 2 com 2 asteriscos, e assim por diante).

PARTE 3 — (Exercícios 31 a 50)

Escopo de variáveis (local/global), parâmetros nomeados fora de ordem, funções que chamam outras funções, laços e validações dentro de funções.

Exercício 31

Arquivo: lista4exercicio31.py

Crie uma variável global `CONTADOR = 0` e uma função `incrementar_contador()` que, usando a instrução `global`, some 1 ao contador a cada chamada. Chame a função 5 vezes e exiba o valor final do contador.

Exercício 32

Arquivo: lista4exercicio32.py

Crie uma função `media_ponderada(nota1, nota2, peso1=1, peso2=1)` que retorne a média ponderada entre as duas notas. Teste chamando a função com parâmetros nomeados, em ordem diferente da declaração (ex: `media_ponderada(peso2=3, nota1=7, nota2=9, peso1=2)`).

Exercício 33

Arquivo: lista4exercicio33.py

Crie uma função `eh_primo(n)` que retorne `True` se o número for primo e `False` caso contrário (utilize um `for` dentro da função para testar os divisores).

Exercício 34

Arquivo: lista4exercicio34.py

Crie uma função `fatorial_iterativo(n)` que calcule o fatorial de um número utilizando `while` (sem recursão).

Exercício 35

Arquivo: lista4exercicio35.py

Crie uma função `mdc(a, b)` que calcule e retorne o Máximo Divisor Comum entre dois números utilizando `while`.

Exercício 36

Arquivo: lista4exercicio36.py

Crie uma função `mmc(a, b)` que calcule o Mínimo Múltiplo Comum entre dois números, chamando dentro dela a função `mdc(a, b)` criada no exercício anterior (fórmula: $mmc = (a * b) / mdc(a, b)$).

Exercício 37

Arquivo: lista4exercicio37.py

Crie uma função `soma_digitos(n)` que retorne a soma dos dígitos de um número inteiro positivo (ex: $123 \rightarrow 1+2+3 = 6$).

Exercício 38

Arquivo: lista4exercicio38.py

Crie uma função `inverter_numero(n)` que retorne o número com os dígitos invertidos (ex: $1234 \rightarrow 4321$).

Exercício 39

Arquivo: lista4exercicio39.py

Crie uma função `conta_vogais(palavra)` que retorne a quantidade de vogais presentes em uma palavra ou frase.

Exercício 40

Arquivo: lista4exercicio40.py

Crie uma função `eh_palindromo(palavra)` que retorne `True` se a palavra for um palíndromo (se lê da mesma forma de trás para frente) e `False` caso contrário.

Exercício 41

Arquivo: lista4exercicio41.py

Crie uma função `validar_senha(senha_correta, tentativas=3)` que, utilizando `while` dentro da função, peça a senha ao usuário até acertar ou esgotar o número de tentativas (padrão 3), exibindo “Acesso liberado!” ou “Acesso bloqueado.”.

Exercício 42

Arquivo: lista4exercicio42.py

Crie uma função `classificar_imc(imc)` que retorne a classificação (Abaixo do peso, Peso normal, Sobrepeso ou Obesidade). Crie também uma função `calcular_imc(peso, altura)` que calcule o IMC e chame `classificar_imc` internamente, retornando a classificação final.

Exercício 43

Arquivo: lista4exercicio43.py

Reaproveite a função `celsius_para_fahrenheit(c)` do exercício 17 e crie uma função `gerar_tabela_conversao(inicio, fim, passo=10)` que imprima, chamando a função de conversão dentro de um `for`, uma tabela de Celsius para Fahrenheit entre o início e o fim informados.

Exercício 44

Arquivo: lista4exercicio44.py

Crie uma função `sacar(saldo, valor_saque)` que retorne o novo saldo após o saque, ou uma mensagem de “Saldo insuficiente.” caso o valor solicitado seja maior que o saldo disponível.

Exercício 45

Arquivo: lista4exercicio45.py

Crie uma função `juros_simples(capital, taxa, tempo)` que retorne o montante final ($\text{montante} = \text{capital} * (1 + \text{taxa} * \text{tempo})$).

Exercício 46

Arquivo: lista4exercicio46.py

Crie uma função `juros_compostos(capital, taxa, tempo)` que retorne o montante final

(montante = capital * (1 + taxa) ** tempo).

Exercício 47

Arquivo: lista4exercicio47.py

Crie uma função `contar_multiplos(inicio, fim, multiplo)` que retorne quantos números múltiplos de `multiplo` existem entre `inicio` e `fim` (utilize `for` dentro da função).

Exercício 48

Arquivo: lista4exercicio48.py

Crie uma função `somar_pares(inicio, fim)` que retorne a soma de todos os números pares no intervalo informado.

Exercício 49

Arquivo: lista4exercicio49.py

Crie uma função `somar_impares(inicio, fim)` que retorne a soma de todos os números ímpares no intervalo informado.

Exercício 50

Arquivo: lista4exercicio50.py

Crie uma função `classificar_triangulo(a, b, c)` que, usando `elif` dentro da função, retorne “Não é um triângulo válido”, “Equilátero”, “Isósceles” ou “Escaleno”, conforme as medidas dos três lados recebidos.